

Lecture 3 — Introduction to OOP in Python

- Why OOP? Motivation in Business/FinTech
- Classes and Objects
- Attributes and Methods
- Constructors (`__init__`) and `self`
- Special Methods (`__str__`)
- Practice Problems

Why OOP? Motivation

- Procedural: data and functions are separate (hard to manage)
- OOP: binds data + behavior → easier to scale and maintain
- Business Analogy: Customer has attributes (name, balance) and actions (deposit, withdraw)

```
# Procedural approach
account = {"name": "Ali", "balance": 10000}

def deposit(acc, amount):
    acc["balance"] += amount

deposit(account, 2000)
print(account)
```

What is a Class?

- Blueprint/template for creating objects
- Defines attributes (data) and methods (behavior)
- CamelCase naming convention

```
class Customer:
    def __init__(self, name, account_no, balance=0.0):
        self.name = name
        self.account_no = account_no
        self.balance = balance
```

```
# Create objects
c1 = Customer("Ali", "A001", 5000)
c2 = Customer("Sara", "A002")
```

```
print(c1.name, c1.balance)
print(c2.name, c2.balance)
```

Objects

- Objects = instances of a class
- Each object has its own copy of attributes
- Objects can call methods defined in the class

```
c1 = Customer("Ali", "A001", 5000)
c2 = Customer("Sara", "A002", 8000)

print(type(c1)) # <class '__main__.Customer'>
print(isinstance(c1, Customer)) # True
```

Attributes

- Variables inside a class
- Defined with self inside `__init__`
- Each object can have different attribute values

```
class Customer:
    def __init__(self, name, account_no, balance=0.0):
        self.name = name
        self.account_no = account_no
        self.balance = balance

ali = Customer("Ali", "A001", 5000)
print(ali.name, ali.balance)
```

Methods

- Functions defined inside a class
- First parameter is self (reference to object)
- Methods access/modify object's attributes

```
class BankAccount:
    def __init__(self, owner, balance=0.0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        return self.balance

acc = BankAccount("Ali", 10000)
print(acc.deposit(2000))
```

The self Keyword

- Represents the instance calling the method
- Must be the first parameter in instance methods
- Allows access to attributes and other methods

```
class Demo:
    def __init__(self, value):
        self.value = value

    def show(self):
        print("Value:", self.value)

d = Demo(42)
d.show() # self points to d
```

Constructors (___init___)

- Special method run automatically when object is created
- Used to initialize attributes
- Always named ___init___

```
class Loan:  
    def ___init___(self, principal, rate, years):  
        self.principal = principal  
        self.rate = rate  
        self.years = years
```

```
loan = Loan(100000, 12, 5)  
print(loan.principal, loan.rate)
```


Special Methods (`__str__`)

- `__str__` controls how object is displayed as a string
- Makes `print()` more meaningful for custom classes

```
class BankAccount:
    def __init__(self, owner, balance=0.0):
        self.owner = owner
        self.balance = balance

    def __str__(self):
        return f"Account[{self.owner}] → Balance: {self.balance:.2f}"

acc = BankAccount("Sara", 12000)
print(acc)
```

Practice Problem 1 — Customer Portfolio

- Create Customer class with name and list of accounts
- Add method `total_balance()` → sum of all accounts

```
class Customer:
    def __init__(self, name):
        self.name = name
        self.accounts = []

    def add_account(self, account):
        self.accounts.append(account)

    def total_balance(self):
        return sum(acc.balance for acc in self.accounts)
```

Practice Problem 2 — Loan Class

- Class Loan with principal, rate, years
- Methods: total_payable(), annual_installment()

```
class Loan:
    def __init__(self, principal, rate, years):
        self.principal = principal
        self.rate = rate
        self.years = years

    def total_payable(self):
        return self.principal * (1 + self.rate/100 * self.years)

    def annual_installment(self):
        return self.total_payable() / self.years

loan = Loan(100000, 12, 5)
print(loan.total_payable(), loan.annual_installment())
```

Practice Problem 3 — Highest Balance

- Create multiple BankAccount objects
- Find the account with highest balance

```
accounts = [  
    BankAccount("Ali", 12000),  
    BankAccount("Sara", 18000),  
    BankAccount("John", 15000)  
]  
  
richest = max(accounts, key=lambda acc: acc.balance)  
print("Highest balance:", richest.owner, richest.balance)
```

Wrap-Up & Next Lecture

- Today: Classes, Objects, Attributes, Methods, `__init__`, `__str__`
- Next: Core OOP principles (Encapsulation, Inheritance, Polymorphism)
- FinTech examples: Transactions, Statements, Account Types